

1. 什么是软件需求?简明说明它在整个软件开发过程中的作用。

- (1) **软件需求**是指用户为了解决某个问题或达到某个目标所需要的条件或能力,也是系统或系统构件为了满足合同、标准、规范或其他正式强制性文档而必须满足的条件或拥有的一种能力。
- (2) 在整个软件开发过程中的作用:
 - ① **沟通的桥梁**:它是业务人员(客户/用户)与技术团队(开发/测试)之间达成共识的基础契约。
 - ② **设计的基准**:软件的架构设计、数据库设计和 UI/UX 设计都必须完全围绕需求来展开。
 - ③ **测试的依据**:测试用例的编写和最终的系统验收都是以需求规格说明书为准绳。
 - ④ **项目管理的抓手**:需求决定了项目的范围,直接影响时间表和成本的估算。

2. 如何去看待软件需求工程,叙述需求工程的主要任务。

- (1) 需求工程不能仅仅被看作是“问客户想要什么并记录下来”,而应该被视为一个系统的、严谨的、跨学科的工程化过程。它旨在探索系统“应该做什么”以及“在什么约束下做”。
- (2) 需求工程的主要任务分为两大阵营:
 - ① **需求开发**:
 - **需求获取**:通过访谈、问卷、原型等方式挖掘用户的隐性与显性需求。
 - **需求分析**:对收集到的信息进行梳理,建立系统模型,消除冲突、歧义和不一致性。
 - **需求规格说明**:将分析结果严谨地文档化,形成《软件需求规格说明书》。
 - **需求验证**:与利益相关者共同评审,确保 SRS 准确、完整地反映了他们的真实意图。
 - ② **需求管理**:
 - 在整个项目生命周期中控制需求变更,维护需求基线,建立需求跟踪矩阵,确保每个需求都有对应的代码和测试。

3. 把你在目前或往项目中遇到的与需求有关的问题写出来。判断每个问题属于需求开发还是需求管理,分析它们对项目的影响或造成这些问题的根本原因。

(1) 问题一:手语识别的“实时性”标准模糊

场景:初期只规定了“系统能识别手语并输出”,但在测试时发现模型推理(CNN+LSTM)延迟很高,画面卡顿。

分类:属于需求开发问题(需求获取与分析不充分)。

分析:根本原因是遗漏了量化的性能需求(如:帧率必须 $\geq 24\text{fps}$, 单次动作识别延迟 $\leq 200\text{ms}$)。这对项目的影响巨大,直接导致后期需要花费大量时间去优化模型结构或牺牲精度换取速度。

(2) 问题二:TexInsite 后端接口范围扩展

场景:开发过程中,前端有时需要提出“加个字段”、“把这个接口拆成两个”的要求,导致后端代码不断重构,延期交付。

分类:属于需求管理问题。

分析:根本原因是缺乏严格的变更控制流程(没有基线概念)。影响是打乱了原有的排期,增加了团队的疲劳感,且频繁修改容易引入新的 Bug。

4. 什么是功能需求?什么是性能需求?分别举例说明。

- (1) **功能需求**:规定了系统必须执行的具体动作或行为。即系统“要做什么”。**举例**:SmartEnergyMaster 项目(个人项目)系统必须允许管理员导入包含每日耗电量 CSV 文

件：系统必须能根据历史数据生成月度能耗报表并支持导出为 PDF。

- (2) **性能需求**:属于非功能需求的一种,规定了系统执行某些功能时的速度、响应时间、吞吐量或资源利用率。即系统“做得有多快/多好”。**举例**:当并发用户数达到 1000 时,系统的平均登录响应时间不能超过 2 秒;手语识别算法在主流移动端 CPU 上的内存占用不得超过 150MB。

5. 需求工程中要考虑哪些约束问题?

- (1) **技术约束**:指定的编程语言(如必须用 Java/Spring Boot)、数据库(如 Redis, MySQL)、部署环境(如必须支持 Docker 容器化部署)。
- (2) **业务与管理约束**:严格的交付截止日期、有限的项目预算、团队人员的技能水平。
- (3) **法律与合规约束**:数据隐私保护法、行业安全标准等。
- (4) **环境与硬件约束**:运行设备的物理限制,如边缘计算设备的算力、网络带宽。

6. 不同角色的需求观是否相同?若不同的话述其需求观之间的差异。

不同

- ① **客户/出资人**:关注**业务价值与投资回报**。他们关心系统能否在预算内按时交付,并解决核心业务痛点或带来盈利。
- ② **最终用户**:关注**易用性与效率**。他们不在乎底层架构,只在乎界面是否直观、操作是否便捷、系统是否会崩溃。
- ③ **开发人员**:关注**逻辑的严密性与技术可行性**。他们希望需求是无歧义的、边界清晰的,最好能直接映射到架构设计和代码实现上。
- ④ **测试人员**:关注**可测试性**。他们需要明确的输入、预期的输出以及量化的验收标准,反感“界面应美观”这类主观描述。

7. 不合理的需求会派生哪些问题?

- (1) **过度工程或返工**:开发了用户根本不需要的功能,或者因为理解偏差导致代码全部推翻重写。
- (2) **成本超支与进度延误**:不断变更或模糊的需求很有可能导致项目延期。
- (3) **架构脆弱**:勉强实现不合理的需求,会破坏软件的内聚性,产生技术债。
- (4) **团队士气低落**:开发者陷入无休止的修改中,对项目失去信心。

8. 成功的需求会带来怎样的好处?

- (1) **降低开发成本**:在需求阶段发现并修正错误的成本,远低于在编码或测试阶段修正的成本。
- (2) **提高产品质量**:明确的边界和性能指标让架构设计更合理,测试覆盖更全面。
- (3) **提升用户满意度**:交付的产品真正切中了用户的痛点,所见即所需。
- (4) **项目可控性增强**:为后续的工作分解、工期估算和风险管理提供了可靠的数据支撑。

9. 优秀需求及需求规格说明有哪些特征?

- (1) 一条优秀的单个需求应具备:**完整性、正确性、可行性、必要性、无歧义性、可测试性**。
- (2) 一份优秀的《软件需求规格说明书》应具备:
- **一致性**:需求之间没有逻辑冲突,例如,既要求极高的加密级别,又要求极低的响应时间,可能存在冲突。
 - **可跟踪性**:能够向前追溯到业务目标,向后追溯到设计、代码和测试用例。
 - **按重要性和稳定性划分优先级**:明确哪些是核心 MVP 功能,哪些是锦上添花。
 - **可修改性**:文档结构清晰,有变更履历,便于后续更新。

10. 讨论项目管理中客户(customer)、用户(user)和干系人(利益相关者)(stakeholder)的概念, 论述项目团队、开发组织内外的干系人组成。

(1) 概念:

干系人(Stakeholder/利益相关者):

- **概念:**这是一个**最广泛的伞形概念**。指的是任何能够影响项目(或系统)、或者被项目(或系统)所影响的个人、群体或组织。
- **地位:**客户和用户都是干系人的子集。发现所有的干系人并理解他们的期望, 是需求工程起步的第一站。

客户(Customer):

- **概念:**负责为软件项目**买单、提供资金支持, 或拥有最终验收决定权**的人或组织(有时也被称为 Sponsor/发起人)。
- **特征:**他们可能永远不会去点击软件里的任何一个按钮, 但他们决定了项目的生死。他们的需求通常停留在宏观层面(如:降本增效、占领市场、满足监管)。

用户(User):

- **概念:****直接与软件系统进行交互操作**的最终使用者。
- **特征:**他们关心软件的具体功能、操作流程和界面体验。

(2) **组成:**在项目管理和开发组织内外, 干系人构成了一个错综复杂的网络。我们通常将其分为两大阵营来管理:

① **项目团队与开发组织内部的干系人。**这些人是软件的“缔造者”, 他们的利益诉求往往围绕着技术实现、职业发展和工作负荷。

- **项目经理(Project Manager):**核心诉求是在预算和时间限制内, 平衡各方需求, 成功交付系统。
- **业务分析师(BA)/需求工程师:**负责桥接外部业务诉求与内部技术语言。
- **系统架构师/开发工程师:**关心技术选型(如 Java, Python, 中间件)、架构的可扩展性、代码的可维护性, 以及需求的技术可行性。
- **测试/质量保证人员(QA):**关心系统是否具有可测试性, 需求是否能够转化为明确的测试用例。
- **UI/UX 设计师:**关心系统的人机交互逻辑和前端表现力。
- **开发组织高层(如研发总监):**关心该项目能否形成可复用的技术资产, 或者团队产出比。

② **项目团队与开发组织外部的干系人。**这些人是软件的“需求源泉”或“约束施加者”。

- **直接出资人/高管:**即前文提到的核心“客户”。掌握预算, 关注核心商业指标。
- **各层级终端用户:**系统的直接操作者。在复杂系统中, 用户需要进一步细分(如管理员角色、普通访客角色、VIP 角色等), 他们的需求往往存在内部冲突(比如管理者希望流程越严谨越好, 一线执行者希望流程越精简越好)。
- **领域专家:**比如开发医疗软件时的资深医生。他们可能既不是出资人也不是高频用户, 但他们掌握着不可或缺的行业规则和业务逻辑。
- **运维与支持团队:**软件上线后负责维护的人员。他们强烈要求软件有完善的日志系统(如 Spring Boot 的监控模块)、清晰的报错机制和简单的部署流程(如 Docker)。
- **监管与合规机构:**例如政府部门、行业协会。他们不关心软件是否赚钱, 只关心软件是否违法(如数据是否加密传输、算法是否合规)。